



Server-side Template Injection

I. Giới thiệu về Server-side Template Injection:

Lỗi hỏng Server-side Template Injection (SSTI) xảy ra khi một ứng dụng web sử dụng các template engine để kết hợp giữa nội dung tĩnh và dữ liệu động nhằm hiển thị nội dung đến người dùng. Khi các giá trị nhập vào của người dùng được chèn trực tiếp vào template thay vì truyền dưới dạng dữ liệu, kẻ tấn công có thể khai thác bằng cách chèn các chỉ thị nguy hiểm vào mẫu, dẫn đến việc thực thi mã độc trên server.

II. Mức độ nguy hiểm của SSTI:

Lỗi hỏng SSTI thường rất nguy hiểm, vì nó mở ra cơ hội cho kẻ tấn công thực thi mã từ xa (Remote Code Execution - RCE) trên server. Nếu tấn công RCE thành công, kẻ tấn công có thể giành quyền kiểm soát server, đánh cắp dữ liệu, và tiếp tục thực hiện các tấn công khác như lây nhiễm vào cơ sở hạ tầng nội bộ. Ngay cả khi không đạt RCE, SSTI vẫn có thể bị khai thác để đọc dữ liệu nhạy cảm hoặc truy cập vào các tệp quan trọng trên server.

III. Cách thức hình thành SSTI

Template Engine và Lỗi hỏng SSTI:

Template engine là công cụ cho phép kết hợp giữa nội dung cố định và dữ liệu động để tạo ra các trang HTML. Một số template engine phổ biến gồm Jinja2 (Python), Twig (PHP), ERB (Ruby), và Freemarker (Java). SSTI xảy ra khi dữ liệu từ người dùng được nối trực tiếp vào một template thay vì được xử lý và truyền vào dưới dạng dữ liệu.

Ví dụ về lỗi gây ra SSTI:

Xem xét đoạn mã sau, nơi tên người dùng được lấy từ tham số GET của URL và nối vào template:



```
$output = $twig->render("Dear " . $_GET['name']);
```

Trong ví dụ này, thay vì truyền tên người dùng dưới dạng dữ liệu vào template, mã này nói trực tiếp biến `name` vào chuỗi mẫu. Điều này mở ra khả năng cho kẻ tấn công truyền các chỉ thị template nguy hiểm thông qua URL, chẳng hạn:

```
http://vulnerable-website.com/?name={{7*7}}
```

Nếu kết quả là "Dear 49", điều này chứng tỏ mẫu đang đánh giá phép toán bên trong `{{ }}`, cho thấy khả năng tồn tại lỗ hổng SSTI.

Những ngữ cảnh dễ bị SSTI:

1. Plaintext context (Ngữ cảnh văn bản thuần):

Khi nội dung từ người dùng được kết hợp trong template theo dạng HTML hoặc văn bản, có thể thử đưa vào các phép tính toán học (chẳng hạn như `{{7*7}}`) để xem liệu kết quả có được tính toán server-side hay không. Nếu kết quả là 49, đây là dấu hiệu của lỗ hổng SSTI.

Code context (Ngữ cảnh mã):

Trong ngữ cảnh mã, các biến điều khiển trong template có thể được truyền thông qua các tham số từ người dùng, dẫn đến việc thực thi sai lệnh. Chẳng hạn, một template có dòng mã:

```
greeting = getQueryParameter('greeting')
engine.render("Hello {"+"greeting+"}", data)
```

2. Với URL: `http://vulnerable-website.com/?greeting=data.username`, đầu ra sẽ là "Hello Carlos" (nếu `username` là Carlos), cho thấy giá trị của `greeting` đang được đánh giá server-side.

IV. Phương pháp khai thác lỗ hổng SSTI

1. Phát hiện lỗ hổng:

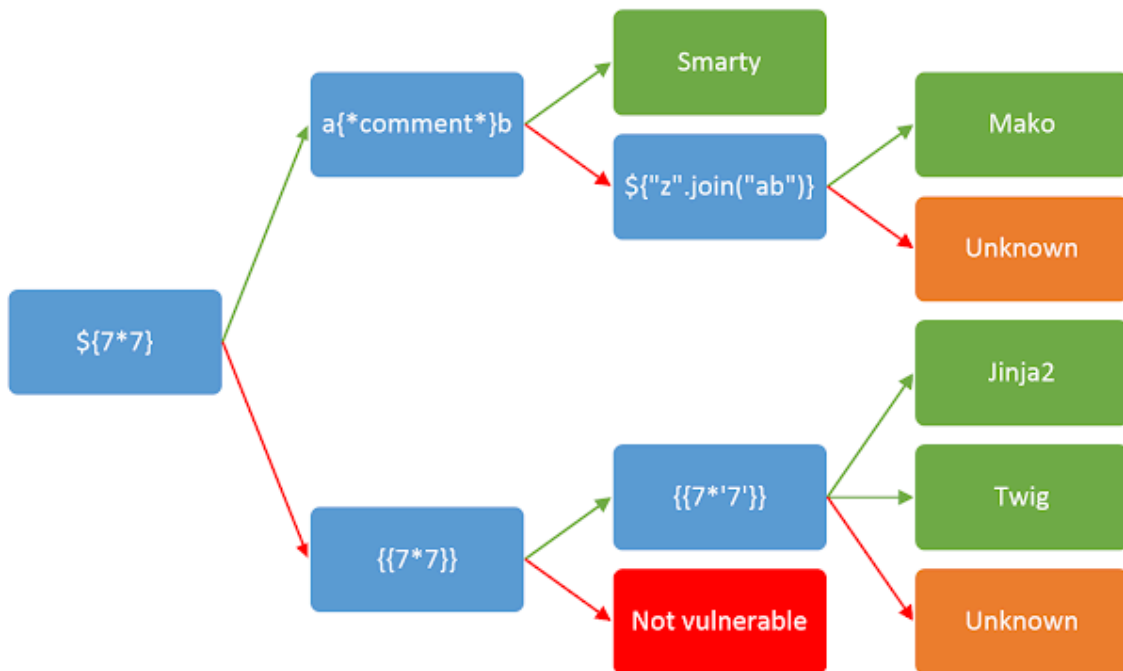
- Kiểm tra bằng cách đưa vào các ký tự đặc biệt, chẳng hạn như `{{`, `[%`, hoặc `}}`. Nếu server trả về lỗi hoặc cho thấy đầu ra có dạng khác thường, có thể lỗ hổng SSTI tồn tại.
- Thử nghiệm thêm với các phép toán để xác nhận, ví dụ: http://vulnerable-website.com/?name={{7*7}}. Nếu đầu ra là 49, điều này xác nhận template đang đánh giá biểu thức bên trong `{{ }}`.

2. Xác định Template Engine:

- Một số template engine có cú pháp đặc trưng. Bằng cách gửi các payload với cú pháp khác nhau, ta có thể xác định engine nào đang được sử dụng.
- Ví dụ:
 - `{{7*7}}` có thể trả về 49 trong Jinja2 và Twig.
 - `{%=foobar%}` có thể gây lỗi và tiết lộ tên engine ERB (Ruby).

3. Khai thác:

- Sau khi xác định được template engine, ta có thể sử dụng các payload tùy biến để thực thi mã trên server. Ví dụ:
 - Với Jinja2, có thể khai thác bằng cách sử dụng các biểu thức của Python để thực thi mã độc hại.



V. Biện pháp phòng chống SSTI

1. Hạn chế quyền tùy chỉnh template từ người dùng:

Không nên cho phép người dùng thông thường tự do tạo hoặc sửa đổi các template. Điều này nên chỉ áp dụng cho những user được tin cậy hoặc với một cơ chế kiểm soát nghiêm ngặt.

2. Sử dụng các template engine "không logic" (logic-less):

Các engine như Mustache không hỗ trợ mã logic bên trong template, chỉ cho phép hiển thị dữ liệu đơn giản. Điều này hạn chế khả năng tấn công qua SSTI vì không có khả năng thực thi mã phức tạp.

3. Sandboxing và Cô lập:

Nếu phải sử dụng các engine cho phép logic, hãy thực thi chúng trong một môi trường sandboxed như Docker với các quyền hạn chế để giảm thiểu rủi ro.

4. Kiểm tra đầu vào của người dùng:

Đảm bảo mọi đầu vào từ người dùng đều được lọc và xử lý đúng cách trước khi đưa vào template.

VI. Tham khảo



<https://portswigger.net/web-security/server-side-template-injection>
<https://viblo.asia/p/server-side-template-injection-vulnerabilities-ssti-cac-lo-hong-ssti-phan-1-W13VM87DLY7>

Mọi thắc mắc hay ý kiến đóng góp vui lòng liên hệ :

Email : r3dctf.info@gmail.com

@Design by R3D CTF Team